

OmniHost.ai — Product & Engineering Spec

2026-09-20 · @Someone

Vision & Product Overview

OmniHost.ai is an AI-chatbot-first operations platform for multi-outlet hotel, restaurant, bar and QSR chains. Every internal role — from a Purchase Head raising a PO to a kitchen hand marking a dish out of stock — works the system by conversing with a role-aware AI chatbot, not by hunting through menus and forms. Conventional screens exist as a fallback, an audit trail, and the surface for anything too visual or fiddly for chat (recipe costing grids, table-layout editing) — never the primary interaction model.

The platform ships two client surfaces. A React + MySQL web application carries the central HQ console, the site back-office, and every fixed POS/KDS/CDS terminal. A Flutter + SQLite mobile app carries guest ordering and on-the-floor staff use, built offline-first so a site keeps taking and serving orders through a network drop and syncs once it's back.

Three organizational layers recur through this spec:

- **App** — OmniHost.ai's own SaaS operators (AppAdmin, AppConfig, AppSupport) who configure and support the platform for every chain that runs on it.
- **Central** — a chain's head office: IT, Revenue Assurance, Quality Assurance, Maintenance, Controls, Purchase, Store, Operations, Culinary, MDM and Marketing, each with a Head and a Team.
- **Site** — the same functions minus MDM, running one outlet as a Team reporting to a Site Head, with a dotted line to their Central Head.

Licensing (Silver/Gold/Platinum — see Licensing Tiers) gates how much of a module a chain gets, not which roles exist: every role in this spec is available at every tier, and the tier decides depth — AI-based indenting, for instance, is a Platinum capability layered on a Silver-and-up manual workflow.

From here, this spec moves outside-in: the role/permission model and the chatbot layer every role sits behind; the guest-facing order lifecycle and floor operations (tables, terminals, POS, KDS, CDS, payments); back-of-house supply chain (menu compliance, recipes, procurement, inventory, waste); people and data operations (staffing, shifts, MDM, COGS); chain-wide functions (content, dashboards, incidents, marketing); and

finally the commercial, platform and delivery layers (licensing, admin, data architecture, non-functionals, phasing) — closing with the assumptions made here that need your sign-off.

System Architecture Overview

OmniHost.ai runs as a three-tier deployment: one App-level control plane (the SaaS layer), many Central tenants (one per chain), each with many Sites (outlets). Every module in this spec is scoped by that hierarchy — a query, a permission, a chatbot action all carry tenant + site context.

Two client surfaces sit on a shared service layer:

- **Web (React + MySQL)** — the central HQ console, every site back-office screen, and all fixed terminals: POS, KDS, CDS, and admin/config screens. React talks to a domain API layer backed by MySQL as the system of record.
- **Mobile (Flutter + SQLite)** — guest ordering and on-the-floor staff use (waiter tablets, kitchen handhelds, store scanning). Built offline-first: SQLite holds a local working set (menu, open orders, stock counts relevant to that device), and a sync engine reconciles to central MySQL over the domain API on reconnect — last-write-wins on non-critical fields, queued-conflict review (surfaced to the Site Head via the chatbot) on anything money- or stock-affecting.

A chatbot/AI gateway fronts both surfaces rather than sitting beside them: every role-driven action — from "raise a PO for 20kg tomatoes" to "mark table 12 ready" — passes through it. The gateway resolves intent, checks it against the caller's role, site scope and license tier, then either calls a domain service directly or renders a structured card (a form, grid or table embedded in the chat thread) when the input is too complex for free text. Chatbot Interaction Model covers this layer in depth; the point here is that it's infrastructure, not a bolt-on — POS, inventory, procurement and every other module expose the same domain APIs to both the chatbot and the fallback screens.

```
flowchart LR
    WEB["React Web App<br/>Central · Site · POS/KDS/CDS"]
    MOB["Flutter Mobile App<br/>Guest + floor staff"]
    GATE["Chatbot/AI Gateway<br/>NLU · permissions · routing"]
    SVC["Domain Services<br/>Orders · Inventory · MDM · more"]
    DB["MySQL - Central"]
    SQL["SQLite - Device"]
    WEB --> GATE
    MOB --> GATE
    MOB --> SQL
    GATE --> SVC
    SVC --> DB
    SQL -- sync on reconnect --> SVC
```

Domain services are organized by module, each owning its own tables and exposing them through the same API layer to web, mobile and the chatbot gateway: Identity & Access, Order/POS/KDS/CDS, Table & Terminal, Payments, Menu/Article, Raw Material & Recipe, Indenting/Procurement, Inventory (receiving/transfer/waste), Site/Staff/Shift, MDM, COGS, Content, Dashboards/Analytics, Ticketing, Licensing, and Marketing.

The API layer's own language/framework isn't specified in the brief; this spec treats it as a stateless REST/GraphQL service tier so the choice (Node.js, Java/Spring, .NET) doesn't change anything else here — see Assumptions & Open Questions. Fixed POS/KDS/CDS terminals run the React app against a live connection by default; Non-Functional Requirements covers site-local resilience options for connectivity drops.

Roles, Org Model & Permission Matrix

Every role sits in one of three layers (Vision & Product Overview): **App** (platform-wide, cuts across all chains), **Central** (one chain's head office), **Site** (one outlet). Central functions run Head + Team; Site mirrors the same functions as a Team reporting solid-line to its Site Head and dotted-line to the matching Central Head. MDM has no site presence — master data is authored and owned centrally only.

Function	Central Head	Central Team	Site Team
IT	IT Head	IT Team	Site IT Team
Revenue Assurance	Revenue Assurance Head	Revenue Assurance Team	Site Revenue Assurance Team

Function	Central Head	Central Team	Site Team
Quality Assurance	Quality Assurance Head	Quality Assurance Team	Site Quality Assurance Team
Maintenance	Maintenance Head	Maintenance Team	Site Maintenance Team
Controls	Controls Head	Controls Team	Site Controls Team
Purchase	Purchase Head	Purchase Team	Site Purchase Team
Store	Store Head	Store Team	Site Store Team
Operations	Operations Head	Operations Team	Site Operations Team
Culinary	Culinary Head	Culinary Team	Site Culinary Team
MDM	MDM Head	MDM Team	— (central-only)
Marketing	Marketing Head	Marketing Team	Site Marketing Team

Two roles sit outside this grid: **Site Head**, who owns everything at one outlet and is the site-level escalation point for every Site Team above; and the **App layer** — AppAdmin, AppConfig, AppSupport — who administer the platform itself, not any one chain (Admin & Configuration Model).

Permission pattern, applied consistently and re-stated only where a module departs from it: Central Heads *configure policy and approve above a threshold* for their function; Central and Site Teams *execute and propose*; a Site Head *approves at site level* within budget/policy set centrally; AppAdmin/AppConfig *configure the platform*, never chain data. Revenue Assurance, Quality Assurance and Controls are cross-cutting check functions — their Central and Site Teams approve or flag transactions raised by *other* functions (a Purchase-raised receipt, a Culinary-raised waste write-off) rather than owning a module of their own; each module section below states exactly which role executes, which approves, and which only views.

Authentication is configurable per chain, set by AppConfig: SSO (SAML/OIDC against the chain's own identity provider) or OmniHost-native username/password with OTP. Role and permission resolution work identically either way once a session exists — login method never changes what a role can do. Security & Non-Functional Requirements covers session policy, MFA and audit in full.

Chatbot Interaction Model

The chatbot is the primary way every internal role gets work done — not a search box bolted onto a dashboard. On login, the gateway resolves the session's role, site/central scope and license tier into a bounded *tool registry*: the specific set of actions (raise indent, approve PO, log wastage, publish content, view PnL) that identity is allowed to invoke. The chatbot can only ever offer what's in that registry, so permission is enforced the same way whether the request comes by typing, tapping a suggested chip, or voice.

Conversation patterns:

- *Direct command* — a fully-specified request executes immediately: "86 the paneer tikka at Site 12."
- *Slot-filling* — a request missing required parameters gets asked for them one at a time: "raise an indent" → bot asks article, quantity, needed-by date if not already given.
- *Structured cards* — anything too dense for text (a multi-line PO, a recipe BOM, a shift roster, a table layout) renders as a form, grid or table inline in the chat thread; the user edits it there rather than switching to a separate screen. This is the "fallback UI" from Vision & Product Overview — it lives inside the chat, not beside it.
- *Confirm-before-commit* — anything financial or stock-affecting (a PO, a stock receipt, a waste write-off, a price change) shows a summary card and waits for an explicit confirm before it posts.

Guardrails:

- Role-permission checks run server-side against the tool registry, never inferred from the model's own judgement — a jailbroken prompt cannot grant an action the role doesn't hold.
- Maker-checker actions route the confirmation card to the *approver's own chatbot* as an actionable item, not to the requester — e.g. a Store-raised stock receipt appears in the site Revenue Assurance Team's chat for sign-off (Stock Receiving, Transfer, Stock-Out & Waste Management).
- Every chatbot-initiated transaction is audit-logged: who, role, site, before/after state, timestamp — the same log a screen-driven action would produce, since the chatbot calls the identical domain API.
- Low-confidence intent or an unsupported request opens a ticket to AppSupport (or the relevant Central function) instead of guessing, and tells the user it's done that.

- Language is configurable per user or site (chain-wide roll-out often spans regional languages on the floor); mobile supports voice input for hands-busy roles — Culinary and Store in particular.

sequenceDiagram

```

participant U as Role user
participant C as Chatbot Gateway
participant P as Permission Check
participant S as Domain Service
U->>C: "Raise a PO, 20kg tomatoes, Site 12"
C->>P: Intent + role + site + tier
P-->>C: Allowed; needs Purchase Head OK above Rs 5,000
C->>U: Confirmation card (qty, rate, vendor, total)
U->>C: Confirm
C->>S: Create PO (pending approval)
S-->>C: PO #2044 created, routed to approver
C->>U: "PO #2044 raised, awaiting Purchase Head approval"

```

Representative intents by role (illustrative, not exhaustive — each module section lists the intents specific to it):

Role	Example chatbot intents
AppAdmin	"Onboard chain X on Platinum"; "Disable SSO for chain Y"
Purchase Team	"Raise indent for tomatoes, 20kg, Site 12"; "Status of PO #2044"
Purchase Head	"Approve PO #2044"; "Open POs above Rs 50,000 this week"
Store Team	"Receive GRN against PO #2044"; "Stock on hand, paneer, Site 12"
Culinary Team	"86 the paneer tikka, Site 12"; "Log 2kg wastage, spoiled tomatoes"
Revenue Assurance Team	"Approve stock receipt GRN-1042"; "Flag cash variance, Site 9"
Quality Assurance Team	"Raise a QA ticket, Site 4, undercooked complaint"
Maintenance Team	"Log AC breakdown, Site 7, kitchen line 2"

Role	Example chatbot intents
Site Head	"Today's ATV vs target, Site 12"; "Approve shift swap, [staff]"
Marketing Team	"Push 20% weekday lunch offer, Sites 3-8"

On the kitchen line specifically, the chatbot coexists with one-tap physical/on-screen buttons (Kitchen Display System) — "order ready" is too time-critical to type, so KDS exposes it as both a chat intent and a button that fires the same underlying action.

Guest Ordering — End-to-End Order Flow

An order is one guest-facing transaction that can span multiple outlets in one chain location (e.g. a resort's bar and its restaurant kitchen in one sitting). Internally it's a parent **Order** holding one **Outlet Sub-Order** per outlet, each with its own line items, tax and service-charge computation — outlet-wise accounting — rolled up into a single **Settlement** the guest pays once. A Sub-Order moves through acceptance, kitchen and serving independently of its siblings, so rejecting or refunding one outlet's lines never touches another's.

flowchart TD

```

    A["Order built<br/>staff or guest app, 1+ outlets"] --> B["Preferences
    applied<br/>set by Culinary config"]
    B --> C["Consolidated bill<br/>outlet-wise lines, one total"]
    C --> D{"Payment method"}
    D -->|Card/UPI| E["Paid online"]
    D -->|Cash| F["Routed to cash counter"]
    F --> G["Cash collected"]
    E --> H["Released per outlet"]
    G --> H
    H --> I{"Outlet counter:<br/>accept or reject"}
    I -->|Accept| J["Queued to kitchen<br/>+ prep-time SLA timer"]
    I -->|Reject| K["Refund initiated<br/>that outlet's lines only"]
    J --> L["Kitchen: in prep"]
    L --> M["Kitchen marks ready"]
    M --> N["Shown on Customer Display System"]
  
```

Step detail:

1. **Order taking.** Staff (POS/Tab/Kiosk) or the guest (mobile app) builds the order for one or more outlets. Guest preferences — dietary flags, spice level, recurring exclusions — are configurable by the Culinary Team per chain or per site, and once set, apply automatically to what's offered and to the kitchen ticket. Every finished-goods item shown at this step carries FSSAI-mandated calorie, allergen and veg/non-veg information (Menu/Article Lifecycle & FSSAI-Compliant Display).
2. **Consolidated bill.** Each outlet's lines are priced and taxed under that outlet's own configuration, then combined into one bill the guest sees and settles once, even though downstream accounting keeps every outlet's figures separate (feeds Dashboards & Analytics PnL by outlet).
3. **Payment.** Card/UPI settles online immediately. Cash routes the guest to a cash counter; once collected there, the order is released the same way an online payment would release it. No sub-order reaches an outlet counter unsettled.
4. **Outlet acceptance.** Each outlet counter accepts or rejects its own sub-order. Rejection immediately initiates a refund scoped to that outlet's lines only — the rest of the guest's order is unaffected.
5. **Kitchen queueing.** An accepted sub-order reaches the kitchen with full line detail and preferences, plus a prep-time deadline timer. The deadline is configurable at chain level (default SLA) and overridable per site by whoever holds site configuration rights (Site Head, within limits AppConfig sets) — not hardcoded. A timer breach raises a flag on the Operations dashboard rather than silently expiring.
6. **Ready for pickup.** The kitchen marks the order ready (chat intent or KDS button, see Chatbot Interaction Model); it then appears on the Customer Display System for the guest, and on Operations/Site dashboards as a closed ticket.

Refunds (step 4) follow the reverse of whichever payment path was used — card/UPI reverses to the original instrument, cash is returned at the counter — and are logged the same as any other financial transaction for Revenue Assurance reconciliation (Payments & Settlement).

POS, KDS & CDS Operations

One exception to "the chatbot is the interface": high-speed order entry at a live counter stays a tap-based POS/Tab/Kiosk UI, because dictating a 12-item order in words is slower than tapping it during a lunch rush — the same domain APIs back both. The chatbot governs everything *around* that fast path: configuration, approvals, exceptions, reporting. Kitchen and customer-display actions follow the same split (Chatbot Interaction Model).

POS operations (industry-standard F&B, all outlet-scoped per Guest Ordering — End-to-End Order Flow):

- Order entry with modifiers, combos and set menus; hold/fire course timing.
- Discounts and comps, gated by role and threshold — above a configured limit, the discount posts as pending until the approver confirms it via chatbot.
- Split bill: by guest, by item, or even split across payment methods within one settlement.
- Void and return, always routed to Revenue Assurance for sign-off (Stock Receiving, Transfer, Stock-Out & Waste Management covers the same pattern for stock).
- Shift open/close with cash-drawer reconciliation; variances raise an RA-visible flag rather than blocking close-out.
- Day-end Z-report per outlet and rolled up per site, feeding Dashboards & Analytics.

Stock-out ("86") propagation. When Culinary marks an article unavailable — by chat intent or a one-tap KDS/POS control — it's blocked in real time on every order-booking terminal for that item's scope (one outlet, one site, or chain-wide, as set when it's raised): POS, Tab, Kiosk and the guest mobile app all stop offering it within seconds. It clears automatically when Store confirms restock or at the next configured reset, whichever the site's policy specifies.

Kitchen Display System (KDS). Tickets route to the right prep station (grill, bar, dessert, etc.) the moment an outlet counter accepts a sub-order, carrying every modifier, allergen flag and guest preference from Guest Ordering. Each ticket shows the prep-time SLA countdown set for that site; a breach flags the ticket and surfaces on the Operations dashboard. Stations advance a ticket (new → in prep → ready) with a single tap or the equivalent chat intent, and can recall a ticket marked ready in error. Where one central kitchen serves several outlets at a site, routing is configurable per article rather than fixed to one kitchen.

Customer Display System (CDS). Guest-facing screens (counter-mounted or on-table) show live order status — queued, preparing, ready — against the guest's token or order number, plus the FSSAI-mandated calorie, allergen and veg/non-veg information for whatever's on order (Menu/Article Lifecycle & FSSAI-Compliant Display). A configurable content slot carries promotions pushed from Content Management or Marketing between order updates, and the screen follows the site's configured display language.

Table Management & Multi-Terminal Order Booking

Table management. Each outlet has a floor layout — tables, capacity, section — authored visually (a floor-plan editor is one of the few genuinely visual fallback screens, alongside recipe costing and shift rosters). Day-to-day table *state* (available → seated → ordered → billing → cleaning), merges for large parties, and section reassignment are chat-first for waitstaff on mobile: "seat 4 at table 12," "merge 9 and 10," "table 6 needs bussing." Reservations and a waitlist aren't in the original brief but are standard for this category; Assumptions & Open Questions flags them as a proposed addition rather than assuming scope.

Multi-terminal order booking. POS, waiter Tab, guest Kiosk and the guest mobile app all create orders through the same Order API from Guest Ordering — End-to-End Order Flow, tagged with their originating terminal for reporting and reconciliation, so an order behaves identically downstream regardless of where it started:

Terminal	Typical user	Notes
POS (counter)	Order-taking staff	Full menu, discounts, splits, cash/card/UPI
Tab (handheld)	Waitstaff, table-side	Tied to a table; syncs to POS/KDS instantly
Kiosk	Guest, self-order	No staff login; simplified flow, integrated payment, same FSSAI display rules
Mobile app	Guest	Browse-and-order UI (Guest Ordering); optional conversational assistant for recommendations, dietary filters and order-status questions

A kiosk or guest-app order still passes through the same acceptance step at the outlet counter (Guest Ordering, step 4) — self-service shortens order *entry*, not the operational flow behind it.

Payments & Settlement

Method	Flow	Settles
Card / UPI	Captured online at order time via a payment gateway	Immediately

Method	Flow	Settles
Cash	Guest routed to a cash counter (Guest Ordering, step 3); order releases once collected	On collection
Room / folio (hotel properties)	Posted to the guest's room folio instead of paid at the outlet	At checkout — industry-standard for hotel F&B; not in the original brief, flagged in Assumptions & Open Questions as a proposed addition

A payment gateway integration is assumed for card/UPI (provider left open — Assumptions & Open Questions) rather than a specific vendor, since the brief doesn't name one.

Settlement vs. accounting. The guest pays the consolidated bill once; the payment record then splits back to each Outlet Sub-Order for accounting, so PnL, COGS and tax reporting stay outlet-wise even though collection wasn't (Guest Ordering — End-to-End Order Flow, Dashboards & Analytics).

Refunds. Triggered by an outlet rejecting its sub-order (Guest Ordering, step 4) or by a post-settlement void/return (POS, KDS & CDS Operations). Card/UPI reverses to the original instrument; cash is returned at the counter. Refunds above a site-configured threshold need Revenue Assurance sign-off via chatbot before they post, mirroring the void approval pattern; every refund is logged against the original transaction for reconciliation.

Service charge & tips. Both are configurable per outlet — rate, whether service charge is a default the guest can remove, and how tips are pooled or attributed to staff (Site, Staff & Shift Management). Defaulting to guest-removable service charge keeps the platform aligned with consumer-protection norms in markets (India included) where a mandatory, non-removable service charge has drawn regulatory scrutiny — the exact rule is left to each chain's own compliance judgement, not hardcoded here.

Menu/Article Lifecycle & FSSAI-Compliant Display

An **article** (a sellable menu item) carries name, category, veg/non-veg flag, tax class, per-outlet price, a linked recipe (Raw Material & Recipe Management), allergen tags, calorie value and serving size, plus images. Price or recipe changes create a new version rather

than overwriting the old one, so a historical order keeps the figures it was actually sold under — required for accurate COGS (Master Data Management (MDM) & COGS).

```
stateDiagram-v2
    [*] --> Draft
    Draft --> PendingReview: Culinary submits
    PendingReview --> Active: Approved + FSSAI fields complete
    PendingReview --> Draft: Sent back
    Active --> Seasonal: Marked unavailable
    Seasonal --> Active: Restored
    Active --> Discontinued
    Discontinued --> [*]
```

An article cannot reach Active without its FSSAI fields filled — that gate is what makes the display requirement below enforceable rather than aspirational.

FSSAI display requirements. FSSAI's clarification on display of information applies to "restaurants with central licenses or ten or more outlets" — which describes essentially every chain OmniHost.ai serves. It requires, on the menu, board or digital display itself: calorie (kcal) content per serving next to each item, alongside the serving size; allergens clearly identified per dish; and veg/non-veg symbols next to items ([Legality Simplified](#), viewed September 2026). Detailed nutrition and organic-ingredient information only needs to be available on request — a handout, booklet or website page — not shown by default on every listing.

The mandatory allergen set to tag per dish: cereals containing gluten, crustaceans, eggs, fish, peanuts, soybeans, milk, tree nuts, and sulphites at 10mg/kg or more ([Auriga Research](#), viewed September 2026).

This spec implements it as: calorie + serving size + veg/non-veg symbol shown inline everywhere an article appears — POS, Kiosk, mobile app, Customer Display System (POS, KDS & CDS Operations) — with a "full nutrition info" expansion (in-app panel or QR to a hosted page) carrying the detailed breakdown for on-request access. If OmniHost.ai ever integrates a delivery aggregator, the same fields export with it, since aggregator listings carry the same obligation.

Ownership: Culinary Team populates recipe-derived calorie and allergen data when authoring an article; MDM Head owns the master field definitions and validates completeness before an article can go Active.

Raw Material & Recipe Management

Raw material master (MDM-owned): name, unit of measure, category, shelf life, storage type, approved vendor(s) and standard cost — the base figures every recipe costs against.

Recipe management is a three-tier bill of materials: raw materials combine into an optional semi-finished good (a batch-prepared base or sauce), and finished goods — the sellable articles from Menu/Article Lifecycle — draw on either directly.

```
flowchart LR
    RM["Raw materials<br/>tomato, onion, spices"] --> SFG["Semi-finished good<br/>tikka masala base"]
    RM --> FG["Finished good<br/>paneer tikka masala"]
    SFG --> FG
```

Every recipe line carries a quantity and an expected yield/wastage percentage (trimming, cooking loss), so the theoretical cost of a finished good is the sum of its components' standard cost, adjusted for wastage — the figure COGS Management reconciles against actual consumption. A recipe version locks to the article version it costs (Menu/Article Lifecycle), so re-costing history stays accurate even after the recipe changes.

Two entry paths, matching the brief's call for both a full tool and a simplified one:

- **Recipe builder** (fallback grid screen) — for multi-level recipes with several semi-finished goods, used when the structure is too deep for chat.
- **Simplified/quick recipe** (chatbot, slot-filled) — for a straightforward finished good: "add a recipe for grilled sandwich: bread 2pc, cheese 30g, butter 10g" walks through ingredient, quantity and unit one at a time and posts a single-level recipe directly.

Culinary Team drafts and edits; Culinary Head approves before a recipe can back an Active article (Roles, Org Model & Permission Matrix). Site-level recipe variants (a spicier version for one region) are proposed by the Site Culinary Team and still need Culinary Head sign-off centrally, keeping costing and FSSAI fields consistent chain-wide.

Indenting, Requisitions & Purchase Orders

An **indent** is an internal request for material from any site or central function (Culinary, Store, Maintenance) against stock or against procurement. A **requisition/PO** is Purchase converting approved need into a vendor order. Both come in a manual and an AI-assisted

form — AI-based indenting and AI-based PO generation are Platinum-tier capabilities (Vision & Product Overview, Licensing Tiers) layered on the same manual workflow everyone else runs.

```

flowchart LR
    A["Team raises indent<br/>chatbot, slot-filled"] --> E["Approved by threshold<br/>Site Head or dept Head"]
    E --> D["AI-suggested indent<br/>from consumption + par level"]
    D -- Platinum --> E
    E --> F["Purchase Team converts<br/>to vendor PO"]
    F --> G{"PO value vs. threshold"}
    G -->|Below| H["Auto-approved"]
    G -->|Above| I["Purchase Head approves"]
    H --> J["PO sent to vendor"]
    I --> J

```

Manual indenting. Raised by chat ("raise an indent for 20kg tomatoes, Site 12, needed tomorrow") or the fallback screen for a multi-line indent. Below a site-configured value it's self-approved by the raiser's Head; above it, the Site Head or the relevant Central Head confirms via their own chatbot before it moves to Purchase.

AI-based indenting (Platinum). The system watches consumption trends against par stock levels per site and article, and proposes an indent before a stockout — "Site 12 will run out of chicken in 2 days at current consumption; raise an indent for 15kg?" — as a confirmation card, never posted without a human accepting it (Chatbot Interaction Model's confirm-before-commit rule applies here too).

Requisition & PO. Purchase Team converts one or more approved indents into a PO against a preferred vendor, pulling rate from the last agreed price or a live quote. Below a configured value the PO auto-approves; above it, Purchase Head confirms (Roles, Org Model & Permission Matrix). AI-based PO suggests vendor and quantity from price history, lead time and minimum order quantities, again as a proposal Purchase Team accepts or edits, not an autonomous commit. Every PO carries the indents it fulfils for traceability into Stock Receiving, Transfer, Stock-Out & Waste Management, where it's matched against what actually arrives.

Stock Receiving, Transfer, Stock-Out & Waste Management

Receiving. Store Team records a GRN (Goods Received Note) against the PO on arrival, checking quantity and quality. Nothing posts to sellable inventory until Revenue Assurance approves — the brief's own requirement — with any shortfall, excess or quality

rejection flagged to Purchase and Quality Assurance rather than silently adjusted.

flowchart LR

```

A["PO"] --> B["Goods arrive at site"]
B --> C["Store creates GRN<br/>qty + quality check"]
C --> D{"Revenue Assurance<br/>approval"}
D -->|Approved| E["Posted to site inventory"]
D -->|Discrepancy| F["Short/excess/reject<br/>flagged to Purchase + QA"]
E --> G["Available for recipes and sale"]

```

Transfer. Moves stock site-to-site or central-to-site. The sending side dispatches (stock leaves its inventory into an in-transit state), the receiving Store Team confirms on arrival before it lands in their own count — the same two-sided confirmation as receiving, without a vendor or RA step, since it never leaves the chain's own book.

Stock-out. Two triggers reach the same result — an article blocked from sale (POS, KDS & CDS Operations): Culinary marks it unavailable directly, or the system marks it automatically the moment a recipe's raw material or the article's own tracked stock hits zero. Either way the block is scoped (outlet, site or chain) and clears the same way — restock confirmed or a configured reset.

Waste management. Any raw material, semi-finished or finished good written off (spoilage, prep loss, a guest return) is logged with a reason code, quantity and its standard cost. Above a site-configured value the write-off needs Revenue Assurance co-sign, mirroring the void/refund pattern (POS, KDS & CDS Operations). Logged waste feeds the variance between theoretical and actual consumption in COGS Management, and rolls up into Dashboards & Analytics so a site's waste trend is visible, not just its total spend.

Site, Staff & Shift Management

Site management. Each site record holds its outlets, assigned license tier, tax jurisdiction, operating hours and the configuration overrides that recur through this spec — prep-time SLA (Guest Ordering), stock-out reset policy (Stock Receiving, Transfer, Stock-Out & Waste Management), service charge default (Payments & Settlement). AppConfig sets what's overridable at all; within that, the Site Head sets the site's own values.

Staff management. A staff record ties one person to a role, a site (or Central), and their chatbot's tool registry follows directly from that assignment (Roles, Org Model & Permission Matrix; Chatbot Interaction Model) — there's no separate permission step once role and site are set.

Shift management. Site Head or a delegated Site Operations Team member builds the roster from shift templates; staff clock in and out by chat intent ("clock me in") or a mobile tap, with attendance logged against the roster for variance (no-show, late, overtime). Shift swaps are requested by the staff member and approved by the Site Head — the example intent from Chatbot Interaction Model. Geo-fenced mobile clock-in is a common addition for multi-outlet chains but isn't in the original brief; noted in Assumptions & Open Questions rather than assumed in scope.

Master Data Management (MDM) & COGS

MDM is the one function with no site presence (Roles, Org Model & Permission Matrix) because its job is a single chain-wide golden record: article, raw material, vendor, tax, UOM conversion, allergen/nutrition reference and site/outlet masters. Other functions propose additions — Purchase a new vendor, Culinary a new raw material — and MDM Team validates and de-duplicates before it's usable anywhere in the chain; MDM Head owns the field definitions themselves (what an article record even contains). This is the data-layer counterpart to Revenue Assurance's transaction-layer checks: neither owns the modules around them, both gate what enters the system of record.

COGS is computed per site and period from the inventory ledger, not just theory:

$$\text{COGS} = \text{Opening Inventory} + \text{Purchases} - \text{Closing Inventory}$$

against which the recipe-derived theoretical cost (Raw Material & Recipe Management) is compared:

$$\text{Variance} = \text{Actual COGS} - \text{Theoretical COGS}$$

A large positive variance points at waste, portioning drift or shrinkage before it shows up as a margin problem — Waste Management's reason-coded write-offs are the main input for explaining it rather than leaving it as an unexplained number. Both figures roll up into the PnL views in Dashboards & Analytics, by article, site and chain.

Content Management

Content — promotions, the Customer Display System's promo slot (POS, KDS & CDS Operations), guest-app banners, in-store signage, staff notices — can originate at a site or centrally, but only centrally-approved content ever goes live.



Site-authored content always routes through the approval step; centrally-authored content is already the approver’s own work and goes straight to scheduling. Either way, publishing targets a scope — one site, a region, or the whole chain — and the distribution step is the same regardless of where the content started, landing on whichever surfaces (CDS, app, signage) the content type is configured for.

Dashboards & Analytics

Three scopes over the same underlying data, not three separate reporting systems:

Dashboard	Audience	Core metrics
Executive	App/Central leadership	PnL by site/chain, ATV, like-for-like sales, license-tier adoption
Site	Site Head, site teams	That site’s PnL, ATV, ticket status, stock status, table turn time
Operations	Operations Head/Team, and each Head’s own cross-site slice	Cross-site comparison on the same metrics — Purchase Head sees spend across every site, Culinary Head sees waste and recipe variance across every site

Baseline metrics from the brief — PnL, ATV, ticket status, stock status — are joined by a few standard additions worth building in from the start: table turn time, order rejection/refund rate, wastage as a percentage of COGS, and labor cost as a percentage of revenue.

Every dashboard is also a chatbot query surface, not only a screen — "today's ATV vs target for Site 12" (Chatbot Interaction Model) resolves against the same aggregates the visual dashboard renders, so the chat answer and the chart are never two different numbers computed two different ways. The screen exists for trend-watching and drill-down; the chat exists for a fast, specific answer.

Incident & Ticket Management

One ticket model serves Revenue Assurance, Quality Assurance and Maintenance, distinguished by category rather than three separate trackers: RA (variance, reconciliation exceptions), QA (food quality or hygiene complaints), Maintenance (equipment or facility faults). A ticket is raised by chat ("log AC breakdown, Site 7, kitchen line 2" — Chatbot Interaction Model) or automatically by the system — a prep-time SLA breach (Guest Ordering), a cash variance at shift close (POS, KDS & CDS Operations), a receiving discrepancy (Stock Receiving, Transfer, Stock-Out & Waste Management) all open tickets themselves rather than waiting for a person to notice.

Every ticket carries a severity-based SLA; breaching it escalates one level — Team to Head, Head to Site Head or Operations — rather than aging silently. Category, site and status are the fields any dashboard or chat query filters on ("open QA tickets, Site 4, this week"), and closure requires the raising role or their Head to verify the fix, not just the assignee to mark it done.

Controls isn't named in the brief with a specific function — this spec treats it as the compliance/audit-exception counterpart to RA's financial checks (policy and process compliance rather than transaction variance) and routes its findings through this same ticket model; Assumptions & Open Questions flags this for confirmation.

Marketing Module

The brief lists Marketing Head/Team at both Central and Site without stating their function; this spec fills the gap with what the role needs to do its job, for confirmation in Assumptions & Open Questions. Core capability: define and schedule offers and campaigns (a discount, a combo, a time-boxed promotion) scoped by article, site and date

range, authored through Content Management's same approval path so a site-level offer still clears Marketing Head before it's live. Offers apply automatically at POS/KDS/CDS when their conditions match an order — no separate coupon-entry step for staff.

A loyalty or points program isn't in the brief; flagged as a common addition rather than assumed in scope. Campaign performance (redemption count, uplift in ATV against a control period) rolls into Dashboards & Analytics so Marketing Head can see what worked, not just what ran.

Licensing Tiers

The brief sets three tiers without allocating features; this is a proposed baseline — confirm and adjust against actual pricing strategy (Assumptions & Open Questions). Every role exists at every tier (Vision & Product Overview); the tier decides depth and whether AI-assisted variants are switched on.

Capability	Silver	Gold	Platinum
Order terminals	POS + Mobile	+ Tab + Kiosk	Same as Gold
Table management	List/status only	Visual floor-plan editor	Same as Gold
Kitchen Display System	Single station	Multi-station routing	+ prep-time auto-tuning
Customer Display System	—	Included	+ personalized promo targeting
Payments	Card / UPI / Cash	+ room-folio posting	Same as Gold
Authentication	Native login only	+ configurable SSO	Same as Gold
Recipe & article management	Manual, single-level	Full 3-tier BOM + versioning	+ AI costing suggestions
Indenting & PO	Manual	Manual, full approval chain	+ AI-based indent/PO suggestions
Receiving, transfer, waste	Basic receiving	+ RA approval workflow, transfer, waste tracking	+ predictive stock-out alerts
MDM governance	Basic master lists	Full approval-gated golden record	Same as Gold

Capability	Silver	Gold	Platinum
COGS	Manual costing	Automated theoretical-vs-actual variance	Same as Gold
Dashboards	Site only	+ Operations (cross-site)	+ Executive, predictive analytics
Incident & ticketing	Manual logging	+ SLA timers, auto-escalation	Same as Gold
Content & marketing	Central publish only	+ site authoring/approval, offers	+ AI-suggested campaign targeting
Chatbot scope	Core transactional intents	+ approvals, dashboard queries, multi-language	+ AI-suggested actions, voice input
Support	Standard AppSupport SLA	Priority SLA	Dedicated support + onboarding

Tier is set per chain (Central), not per site — every site in a chain runs the same tier, since split-tier chains would complicate the permission and data model for no clear benefit; if that's wrong for how OmniHost.ai will actually be sold, flag it and Data Model & Sync Architecture's tenant model changes accordingly.

Admin & Configuration Model

AppAdmin can configure anything on the platform; AppConfig and AppSupport hold whatever slice of that AppAdmin chooses to delegate — delegation is itself a configuration action, grantable and revocable per person, not a fixed second tier of roles.

Capability	AppAdmin	AppConfig (delegated)	AppSupport (delegated)
Platform/infrastructure settings	Yes	No	No
Chain onboarding & license tier	Yes	Typically delegated	No

Capability	AppAdmin	AppConfig (delegated)	AppSupport (delegated)
Feature toggles per chain	Yes	Typically delegated	No
SSO / authentication setup	Yes	Typically delegated	No
Grant scopes to AppConfig/AppSupport	Yes	No	No
View chain data for troubleshooting	Yes	As granted	Typically delegated
Resolve escalated tickets (Chatbot Interaction Model)	Yes	As granted	Typically delegated
Time-boxed access to a chain's account for support	Yes	As granted	Typically delegated, audit-logged

Any AppSupport or AppConfig access into a specific chain's data is scoped, time-boxed and audit-logged the same way an internal approval is (Roles, Org Model & Permission Matrix) — a chain should be able to see exactly who at OmniHost.ai touched their data and why, which matters for a platform holding another company's sales and cost data.

Data Model & Sync Architecture

MySQL is the system of record; every table carries a tenant (chain) id and, where relevant, a site id, scoped at the API layer rather than trusted to the client.

Entity	Key fields	Relates to
Chain / Site / Outlet	tier, tax jurisdiction, config overrides	Owns everything below
User / Role	role code, site/central scope	Drives the chatbot tool registry
Article (versioned)	price, tax class, FSSAI fields, recipe ref	Recipe, Order lines
RawMaterial	UOM, standard cost, vendor	Recipe lines, Stock ledger

Entity	Key fields	Relates to
Recipe	component lines, qty, wastage %	Article, RawMaterial
Order / OutletSubOrder / OrderLine	status, terminal origin	Settlement, Table, Ticket (on breach)
Settlement / Payment	method, amount, refund ref	Order, outlet-wise accounting
Indent / PurchaseOrder / GRN	qty, approver, status	RawMaterial, vendor, Stock ledger
StockLedger	txn type (receipt/transfer/waste/sale), qty, cost	RawMaterial, Site, COGS
Table / Shift / Attendance	state, roster, clock events	Site, User
Ticket	category, severity, SLA, status	Any module that raised it
ContentItem / Campaign	scope, schedule, approval status	Site/Chain, Article (for offers)
ChatbotAuditLog	intent, role, before/after state	Every module (Chatbot Interaction Model)

Mobile (SQLite) mirrors a working subset only — menu/article cache, open orders, table state, the device's own attendance/order actions — not the whole schema. Writes queue in a local outbox; on reconnect, the sync engine replays them against the same domain APIs the web app uses, then pulls deltas (menu changes, stock-outs, price updates) since the device's last sync token, so it never re-downloads data it already has.

Conflict resolution follows the rule from System Architecture Overview: reference data (menu, prices) is last-write-wins from the server, since the server copy is always authoritative once it's back online. Anything financial or stock-affecting — an offline order that turns out to clash with a stock-out that happened while the device was offline — doesn't auto-resolve; it queues for the Site Head to review, with both versions visible, rather than silently picking one.

Non-Functional Requirements

Security. RBAC is enforced server-side against each role's tool registry, never inferred from the chatbot's own output (Chatbot Interaction Model). Authentication is SSO or native + OTP per chain configuration (Roles, Org Model & Permission Matrix). Card data is tokenized at the payment gateway; OmniHost.ai's own database never stores a full card number, which keeps PCI-DSS scope to the gateway integration rather than the whole platform. Every chatbot- or screen-driven transaction is audit-logged with who, role, site, before/after state.

Offline & resilience. The Flutter app is offline-first by design (Data Model & Sync Architecture). Fixed POS/KDS/CDS terminals default to needing a live connection; for sites with unreliable connectivity, a site-local cache (a small local service the terminals fall back to) is worth building rather than assuming always-on network — flagged here as a recommendation, not committed scope.

Scalability & performance. The API tier scales horizontally per tenant load; MySQL read replicas serve dashboards and reporting so a heavy analytics query never slows down live POS/KDS traffic. KDS and CDS updates target sub-second latency from an order-status change to the screen reflecting it — that's the operational moment guests and kitchen staff actually feel.

Localization. Chatbot and UI language is configurable per user or site (Chatbot Interaction Model). Multi-currency and cross-border tax handling aren't in the brief; likely relevant if OmniHost.ai ever serves a chain operating across countries, so it's called out in Assumptions & Open Questions rather than built to a guess.

Compliance & audit. FSSAI field completeness gates an article going Active (Menu/Article Lifecycle & FSSAI-Compliant Display). Financial actions (refunds, discounts, write-offs, PO approval) keep a full audit trail for Revenue Assurance and, ultimately, statutory audit.

Disaster recovery. Central MySQL needs a defined backup cadence and a recovery point/time objective before go-live; specific numbers are a business decision, not assumed here (Assumptions & Open Questions).

Phased Delivery Roadmap

Every module above is specified at full depth; building them in this order keeps each phase shippable and roughly tracks the license tiers, without forcing the commercial and delivery sequencing to match exactly.

Phase	Focus	Key modules	Roughly
1 — Foundation	Single outlet, working end to end	Roles & chatbot gateway core, React/MySQL and Flutter/SQLite skeletons, Table management, POS/KDS/CDS, single-outlet order flow, Card/UPI/Cash payments, Article/Menu with FSSAI fields, Site management	Silver
2 — Supply chain	Back-of-house, multi-outlet	Multi-outlet Sub-Orders/Settlement, Raw Material & Recipe (full BOM), manual Indenting/PO, Receiving/Transfer/Waste with RA approval, MDM governance, COGS, Staff/Shift, Content approval workflow	Gold
3 — Visibility & chain ops	Cross-site management	Site + Operations + Executive dashboards, Incident/Ticketing (RA/QA/Maintenance/Controls), Marketing module, SSO, Admin/Config delegation	Gold-Platinum
4 — AI depth	Platinum differentiation	AI-based indenting/PO, predictive stock-out alerts, predictive analytics, AI-suggested campaign targeting, voice input, room/folio posting, aggregator integration	Platinum

Each phase assumes the one before it is in production, not just built — Phase 2's approval chains, for instance, need Phase 1's role/permission model actually enforced in the field, not merely coded.

Assumptions & Open Questions

Everywhere this spec filled a gap the brief left open, it's flagged in place and collected here for one pass of sign-off.

Proposed additions (confirm to keep, or say which to drop):

- ☐ Table reservations and a waitlist (Table Management & Multi-Terminal Order Booking)

- ☐ Room/folio posting as a payment method for hotel properties (Payments & Settlement)
- ☐ Geo-fenced mobile clock-in for attendance (Site, Staff & Shift Management)
- ☐ A loyalty/points program alongside offers and campaigns (Marketing Module)
- ☐ Site-local resilience caching for fixed POS/KDS/CDS terminals during network drops (Non-Functional Requirements)

Filled gaps (confirm the interpretation is right):

- ☐ Guest ordering keeps a browse-and-tap menu on mobile/kiosk, with an optional chatbot layered on top for recommendations and status — "chatbot is the interface" was scoped to internal roles, not guest checkout (Table Management & Multi-Terminal Order Booking)
- ☐ Controls' function wasn't specified — treated as a compliance/audit-exception counterpart to Revenue Assurance's financial checks (Incident & Ticket Management)
- ☐ Marketing Head/Team's remit wasn't specified — filled in as offers/campaign management (Marketing Module)

Decisions needed before or at build:

- ☐ Silver/Gold/Platinum feature allocation is a proposed baseline (Licensing Tiers) — confirm against actual pricing strategy
- ☐ License tier applies per chain, not per site — confirm split-tier chains are genuinely out of scope
- ☐ Payment gateway vendor (Payments & Settlement)
- ☐ API layer language/framework — doesn't change this spec's content, but engineering needs a choice (System Architecture Overview)
- ☐ Multi-currency / cross-border tax handling, if any chain will operate across countries (Non-Functional Requirements)
- ☐ Backup cadence and recovery point/time objectives (Non-Functional Requirements)